

Lesson 9 — Particles And Juice Engineering

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

A particle

```
{ x, y, vx, vy, life, maxLife, size }
```

```
// update
p.vy += GRAVITY * dt;
p.x += p.vx * dt;
p.y += p.vy * dt;
p.life -= dt;

// draw — the whole visual trick
const t = p.life / p.maxLife; // 1 → 0
ctx.globalAlpha = t;
ctx.fillRect(p.x, p.y, p.size*t, p.size*t);
```

Remove dead ones **backwards**. Reset `globalAlpha = 1` afterwards, or the score goes transparent too.

Random direction

```
const a = Math.random() * Math.PI * 2;
vx = Math.cos(a) * speed;
vy = Math.sin(a) * speed;
```

Random `vx` and `vy` separately **clusters on the diagonals**.

Four effects, one emitter

	dir	speed	gravity	life
explosion	all	high	low	short
dust	up-ish	low	low	medium
sparks	away	high	high	short
smoke	up	tiny	negative	long

Pooling

```
// built once, never allocated again
if (!pool[i].alive) { reuse pool[i]; }
```

Array length never changes. A full pool **drops** the request, which caps the work per frame.

Pool when you have **measured** a reason to. The symptom it fixes is one dropped frame every few seconds, not a slow game.

Screen shake

```
shake = Math.min(MAX, shake + amount); //
cap
shake *= Math.exp(-DECAY * dt); //
decay

ctx.save();
ctx.translate(rand(-shake, shake),
              rand(-shake, shake));
drawWorld();
ctx.restore(); //
OFFSET, not the camera
```

3–8 px for a hit. 15 for something enormous. If you *notice* it, it is too much.

Hitstop — four lines, huge effect

```
if (hitstop > 0) { hitstop -= realDt; dt = 0; }
```

50–80 ms. Under 30 is invisible, over 120 feels like lag.

Tween

```
tween(obj, "flash", 0, 0.12, easeOutQuad);
```

One list, one update, every effect.

Measuring

```
const t0 = performance.now();
work();
samples.push(performance.now() - t0);
```

Report the **average and the worst**. The worst is what the player feels.

Section A — Recall

A1. What three things is a particle made of? What is `t = life / maxLife` used for? [3 marks]

A2. State the 100 ms rule, and say how many frames that is at 60 fps. [2 marks]

A3. Why does picking random `vx` and `vy` not give an even spread of directions? What should you do instead? [3 marks]

A4. What problem does object pooling solve? Describe the *symptom* a player would notice without it. [3 marks]

A5. Name three mistakes people make with screen shake, and what each one looks like. [3 marks]

A6. What is hitstop, how long should it last, and why does it work? [2 marks]

A7. Why report both the average and the worst frame time rather than just the average? [2 marks]

Section B — Predict the output

B1. A particle has `maxLife` 0.8 s and `life` 0.2 s, with `size` 6. What are its alpha and drawn size? [3 marks]

B2. `shake` is 12 and `SHAKE_DECAY` is 6 per second. What is it after 0.2 s? After 1 s? [3 marks]

B3. What goes wrong here, and what does the player see?

[4 marks]

```
for (let i = 0; i < particles.length; i++) {  
  if (particles[i].life <= 0) { particles.splice(i, 1); }  
}
```

B4. The particle loop sets `ctx.globalAlpha` each time and never resets it. The score is drawn afterwards. Describe exactly what the player sees as the last particle dies.

[3 marks]

B5. A pool holds 500 particles. An explosion asks for 300 and another asks for 300 one frame later, while 200 from the first are still alive. How many does the second explosion get, and is that a problem?

[4 marks]

Section C — Find and fix the bug

C1. The particles look like a cloud of hard-edged squares that vanish all at once rather than fading.

[3 marks]

```
ctx.globalAlpha = 1;  
ctx.fillRect(p.x, p.y, p.size, p.size);
```

C2. After a few explosions the view has drifted away from the player and never comes back.

[4 marks]

```
function addShake(n) { camera.x += (Math.random() * 2 - 1) * n; }
```

C3. The screen shakes for the rest of the game after the first explosion.

[3 marks]

```
function updateShake(dt) {  
  if (shake < 0.1) { shake = 0; }  
}
```

C4. Hitstop makes the game feel broken: the whole screen locks up, including the pause menu and the score. [4 marks]

```
if (hitstop > 0) { hitstop -= dt; return; } // skips update AND draw
```

C5. The game runs at a steady 60 fps and stutters noticeably about once a second. The author has measured the average frame time at 4 ms and concluded there is no problem. [3 marks]

Section D — Build it

Use your game from the previous lessons.

- ☐ **Checkpoint 1.** Build one particle emitter. Spawn 20 particles on a key press. Get the fade working with `t = life / maxLife` before you do anything else.
- ☐ **Checkpoint 2.** Make four presets from the same emitter: explosion, dust, sparks and smoke. Smoke must use negative gravity. Put every number on a slider first and find the presets by playing.
- ☐ **Checkpoint 3.** Add screen shake as a decaying, capped offset applied with `translate`. Start at 6 pixels. Then try 40 so that you know what too much looks like.
- ☐ **Checkpoint 4.** Add hitstop on a slider from 0 to 200 ms. Find your own number. Make sure the HUD still draws while it is frozen.
- ☐ **Checkpoint 5.** Add a white flash on the thing that got hit, decaying over about 120 ms, and squash-and-stretch on landing.
- ☐ **Checkpoint 6.** Add floating damage numbers: a position, a value, and a countdown, drifting upwards and fading. It is the particle system with text.
- ☐ **Checkpoint 7.** Build the tween system from the notes, and rewrite your flash and your squash to use it. Count how many timers that removed.
- ☐ **Checkpoint 8.** Add a switch for every effect, and a frame-time readout showing average and worst. Then turn everything off, play for a minute, and turn them on one at a time.

BEFORE YOU TUNE ANYTHING

Get the frame-time readout (average *and* worst over the last two seconds)

working early, and leave it in your game for the rest of the course. Every performance question after this becomes a five-second answer instead of an argument.

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost.

E1. Hitstop freezes the game — but the hit's particles probably should keep moving and the HUD definitely should. Write the rule for exactly what freezes.

E2. At what particle count did pooling start to matter on *your* machine? What does that tell you about when to bother? Would your answer change on a five-year-old school laptop, and how would you find out?

E3. Design three strengths of hit — light, medium, killing blow — using shake, particles, hitstop, flash, squash, sound and numbers. Do you scale everything together, or do some effects only appear for the big one? Give numbers.

E4. The hard one. Juice can lie. A shake and a flash say "something important happened", so using them for everything says nothing, and using them for a hit that did no damage actively misleads. Where is the line? Give one concrete example of juice that would be dishonest.

Stretch goals

1. Trails: the last ten positions, fading.
2. Colour over life: yellow → red → grey.
3. A tween with an `onDone` callback, and three chained effects.
4. Impact direction: particles fly away from where the hit came from.
5. A frame-time graph you keep in your game for the rest of the course.

